

Gary Hobson

MAT260 Module 7

June 16, 2025

13.6-2

Alice is signing $h(m) \bmod 101 = 2m \bmod 101$.

She signs the result of this function, not the message itself. So if someone can find another message m_1 such that:

$$h(m_1) \equiv h(m) \pmod{101},$$

then the same signature is valid for both messages.

Since:

$$h(m) \equiv 2m \pmod{101},$$

it follows that:

$$\begin{aligned} 2m &\equiv 2m_1 \pmod{101} \\ \implies m &\equiv m_1 \pmod{101} \end{aligned}$$

So any m_1 that satisfies $m_1 \equiv m \pmod{101}$ will have the same hash output, and hence will produce a valid signature using the same hash.

Example:

Suppose Alice signed message $m = 10$.

Then:

$$h(10) \equiv 2 \cdot 10 \equiv 20 \pmod{101}$$

Now let's choose another message:

$$m_1 = 10 + 101 = 111$$

Then:

$$h(111) \equiv 2 \cdot 111 = 222 \equiv 20 \pmod{101}$$

So, $m_1 = 111$ will also produce the same hash and reuse the signature originally intended for $m = 10$.

13.6-3

Alice signs the hash of a benign message (petition), and Eve finds a fraudulent message (bank withdrawal) with the same hash due to the small output size. Eve can now forge a valid-looking signature on the harmful message without Alice's knowledge.

So if, Eve prepares many variations of two documents: one Alice will sign (the petition) and one she won't (the bank withdrawal).

She computes hashes for both sets, exploiting the fact that a 60 bit hash function makes birthday collisions highly likely when you generate 2^{30} versions.

Once a hash collision is found, Eve gets Alice to sign the innocent version, then reuses that signature on the fraudulent version both share the same hash, so the signature is valid for both.

This is considered a standard attack when the hash output is too short according to the book.

14.4-4

To determine whether the plaintext is **ATTACKONTHURSDAY** or **ATTACKONSATURDAY**, I have to test which one could have plausibly resulted in the ciphertext **TUCDZARQKERUIZCU**, considering the Enigma's no self encryption rule.

Compare each character in the candidate plaintexts with the corresponding character in the ciphertext.

If any letter in the plaintext matches the same letter in the ciphertext at the same position, that plaintext is invalid under Enigma rules.

Check: **ATTACKONTHURSDAY**

Compare:

- $A \neq T$
- $T \neq U$
- $T \neq C$
- $A \neq D$
- $C \neq Z$
- $K \neq A$
- $O \neq R$
- $N \neq Q$
- $T \neq K$
- $H \neq E$
- $U \neq R$
- $R \neq U$
- $S \neq I$
- $D \neq Z$
- $A \neq C$
- $Y \neq U$

No position has the same letter in both plaintext and ciphertext $\rightarrow$ Valid under Enigma.

Check: ATTACKONSATURDAY

Compare:

- $A \neq T$
- $T \neq U$
- $T \neq C$
- $A \neq D$
- $C \neq Z$
- $K \neq A$
- $O \neq R$
- $N \neq Q$
- $S \neq K$
- $A \neq E$
- $T \neq R$
- $U = U$ (Invalid under Enigma: letter encrypted as itself)
- $R \neq I$
- $D \neq Z$
- $A \neq C$
- $Y \neq U$

The correct plaintext is ATTACKONTHURSDAY, because:

It satisfies the rule that no letter encrypts to itself, which is a structural property of Enigma.

The other candidate, ATTACKONSATURDAY, fails this rule at the 12th character.